

Python Lesson 6

Raspberry Pi Health Check Script - from fake values to live system data

Purpose

This lesson captures the moment the script stopped using made-up static numbers and started pulling real information from the Raspberry Pi. The goal is not to memorize every command. The goal is to own the pattern: imports, definitions, execution, and output.

Core mental model: imports are tools you bring in, definitions are tools you build, and the bottom of the script is where the mission actually executes.

What the Script Does

Hostname	Uses <code>socket.gethostname()</code> to ask Linux for the device name.
IP address	Creates a temporary socket so Linux can reveal which local IP it would use to reach the network.
Available RAM	Reads <code>/proc/meminfo</code> , finds <code>MemAvailable</code> , converts kB to GiB, and rounds it.
Total RAM	Reads <code>/proc/meminfo</code> , finds <code>MemTotal</code> , converts kB to GiB, and rounds it.
CPU cores	Uses <code>os.cpu_count()</code> to return the number of CPU cores.
Lab status	Uses if/else logic to decide whether the Pi meets your lab threshold.

The Structure You Noticed

```
# 1. Imports: bring in outside tools
import os
import socket

# 2. Definitions: create reusable tasks

def get_cpu_cores():
    return os.cpu_count()

# 3. Execution: actually run the tasks
cpu_cores = get_cpu_cores()

# 4. Output: show or use the result
print("CPU Cores:", cpu_cores)
```

Key lesson: def blocks do not run by themselves. They only tell Python what a task means. The task runs only when the bottom of the script calls the function.

Definitions vs Execution

Definition	<code>def check_cpu(cpu_cores):</code> creates a reusable task called <code>check_cpu</code> .
Execution	<code>check_cpu(cpu_cores)</code> actually runs that task.

Return	Sends a value back from a function so another part of the script can use it.
Print	Displays information for the human reading the terminal.
Variable	Stores a value under a name, like available_ram or cpu_cores.

Working Script

Save this as lesson6.py. This is the clean version of the working Raspberry Pi health-check script.

```
from pathlib import Path
import os
import socket

def get_hostname():
    return socket.gethostname()

def get_ip_address():
    try:
        test_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        test_socket.connect(("8.8.8.8", 80))
        ip_address = test_socket.getsockname()[0]
        test_socket.close()
        return ip_address
    except OSError:
        return "Unable to get IP address"

def get_available_ram():
    meminfo = Path("/proc/meminfo").read_text()

    for line in meminfo.splitlines():
        if line.startswith("MemAvailable:"):
            ram_kb = int(line.split()[1])
            ram_gib = ram_kb / 1024 / 1024
            return round(ram_gib, 2)

    return None

def get_total_ram():
    meminfo = Path("/proc/meminfo").read_text()

    for line in meminfo.splitlines():
        if line.startswith("MemTotal:"):
            ram_kb = int(line.split()[1])
            ram_gib = ram_kb / 1024 / 1024
            return round(ram_gib, 2)

    return None

def get_cpu_cores():
    return os.cpu_count()

def display_device_info():
    hostname = get_hostname()
    ip_address = get_ip_address()
    available_ram = get_available_ram()
    total_ram = get_total_ram()
    cpu_cores = get_cpu_cores()
```

```

print("==== Device Information =====")
print("Hostname:", hostname)
print("IP Address:", ip_address)
print("Available RAM:", available_ram, "GiB")
print("Total RAM:", total_ram, "GiB")
print("CPU Cores:", cpu_cores)

def check_ram(ram):
    if ram >= 4:
        print("Enough RAM for lab.")
    else:
        print("Not enough RAM.")

def check_cpu(cpu_cores):
    if cpu_cores >= 2:
        print("Enough CPU cores for lab.")
    else:
        print("Not enough CPU cores.")

def lab_ready(ram, cpu_cores):
    if ram >= 4 and cpu_cores >= 2:
        print("LAB STATUS: READY")
    else:
        print("LAB STATUS: NOT READY")

available_ram = get_available_ram()
cpu_cores = get_cpu_cores()

display_device_info()
check_ram(available_ram)
check_cpu(cpu_cores)
lab_ready(available_ram, cpu_cores)

```

Working Output

```
(enauto) student @enauto-pi:~/projects/python $ python lesson6.py
===== Device Information =====
Hostname: enauto-pi
IP Address: 192.0.2.10
Available RAM: 7.4 GiB
Total RAM: 7.64 GiB
CPU Cores: 4
Enough RAM for lab.
Enough CPU cores for lab.
LAB STATUS: READY
```

That output confirms the script is pulling live system data from the Pi, not using static values.

Errors You Fixed

linestartswith	Fixed to line.startswith(...). Python needs the variable name line, then the method .startswith().
test.socket.close()	Fixed to test_socket.close(). The variable was named test_socket.
avaiable_ram	Fixed to available_ram. Variable spelling must match exactly.
display_device_info(hostname, ip_address, ram)	Fixed to display_device_info(). The function now pulls the data itself and takes no arguments.
NameError	Python raised this when hostname, ip_address, and ram were called before they existed.

Python Reading Pattern

When looking at a script, read it in this order:

- First, identify the imports. These show which outside toolboxes the script uses.
- Second, scan the function definitions. These show what reusable tasks the script knows how to perform.
- Third, go to the bottom. This is usually where the script actually starts doing work.
- Fourth, trace the data. Watch where a value is created, stored, passed into a function, returned, and printed.

From-Memory Drill

Before adding disk space, drill the smaller pattern from memory. Repetition is how this becomes automatic.

```
import os

def get_cpu_cores():
    return os.cpu_count()

cpu_cores = get_cpu_cores()

print("CPU Cores:", cpu_cores)
```

```
import socket

def get_hostname():
    return socket.gethostname()

hostname = get_hostname()

print("Hostname:", hostname)
```

Next Upgrade - Disk Space

Put this on the back burner until the import/definition/execution pattern feels solid. When ready, disk space will use the shutil toolbox.

```
import shutil

def get_disk_space():
    total, used, free = shutil.disk_usage("/")
    free_gib = free / 1024 / 1024 / 1024
    return round(free_gib, 2)
```

Bottom Line

You do not need to memorize every module yet. Own this structure first: imports -> definitions -> execution -> output. Once that map is automatic, adding new capabilities becomes filling in known slots instead of staring at random code.